House Prices

Advanced Regression Techniques

Machine Learning Project Report

Eng: Beshoy Osama

Dataset	Kaggle — House Prices: Advanced Regression Techniques
Train / Test	1,460 / 1,459 samples | 81 features
Task	Regression — Predict residential property sale prices
Metric	Root Mean Squared Log Error (RMSLE)
Best Cross Validation RMSLE	0.1138 (XGBoost) | Kaggle Public: 0.13115

1. Introduction

This report documents a complete machine learning pipeline applied to the Kaggle competition House Prices: Advanced Regression Techniques. The competition challenges participants to predict the final sale price of residential homes in Ames, Iowa, using 79 explanatory variables describing nearly every aspect of the properties.

Objectives:

- Perform exploratory data analysis (EDA) to understand the dataset structure, distributions, and feature relationships.
- Build a robust preprocessing pipeline: missing value imputation, outlier removal, feature engineering, and encoding.
- Train and evaluate multiple regression models: SVR, Linear Regression, Lasso, and XGBoost.
- Compare models using 5-fold cross-validated RMSLE to identify the best-performing approach.

The evaluation metric is **Root Mean Squared Logarithmic Error (RMSLE)**, which penalizes under-predictions more than over-predictions and is well-suited to price data that spans orders of magnitude. All models were assessed exclusively via cross-validation on the training set, since **Kaggle withholds the true test labels**.

2. Dataset

The dataset is publicly available at kaggle.com/competitions/house-prices-advanced-regression-techniques. It contains two files: **train.csv** (1,460 rows, 81 columns including the target SalePrice) and **test.csv** (1,459 rows, 80 columns). Features cover physical attributes, location, quality ratings, and sale conditions.

Split	Rows	Columns	Target
Train	1,460	81	SalePrice
Test	1,459	80	— (held out by Kaggle)

Feature Types

Type	Count	Examples
Categorical (object)	43	Neighborhood, MSZoning, GarageType, RoofStyle
Integer	35	OverallQual, YearBuilt, GrLivArea, BedroomAbvGr
Float	3	LotFrontage, MasVnrArea, GarageYrBlt

3. Exploratory Data Analysis (EDA)

3.1 Target Variable — SalePrice

The raw SalePrice distribution is right-skewed (skewness ≈ 1.88), violating the normality assumption required by several regression models. Applying a **log(1 + x) transform** reduces the skewness to ≈ 0.12, producing a near-normal distribution and stabilising variance across the price range.

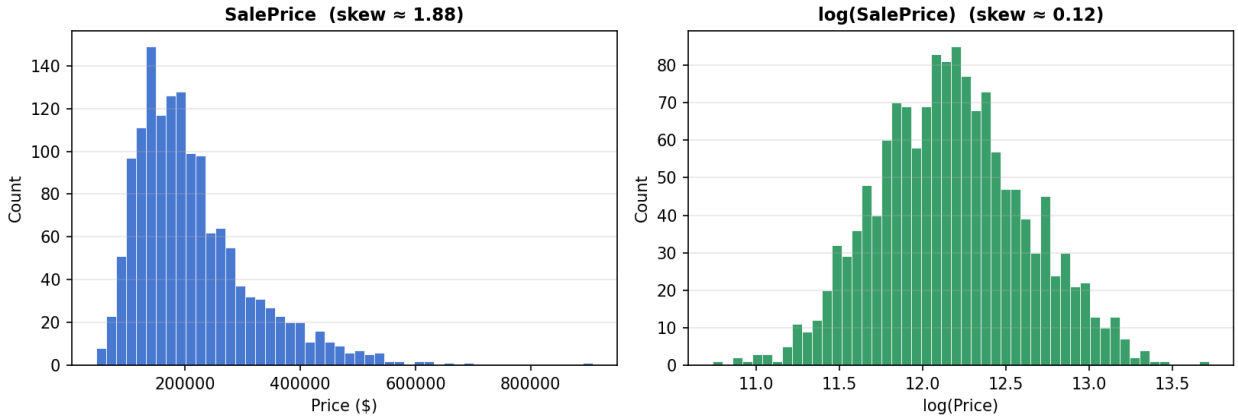

Figure 1 — SalePrice before (left) and after (right) log-transform.

3.2 Missing Values

The training set contains **7,829 total missing values** across 19 columns. Four columns — PoolQC (99.5%), MiscFeature (96.3%), Alley (93.8%), and Fence (80.8%) — are almost entirely empty and provide negligible predictive signal. These were dropped. The remaining missing values reflect a genuine absence of a feature (e.g. no garage, no basement) rather than data collection errors.

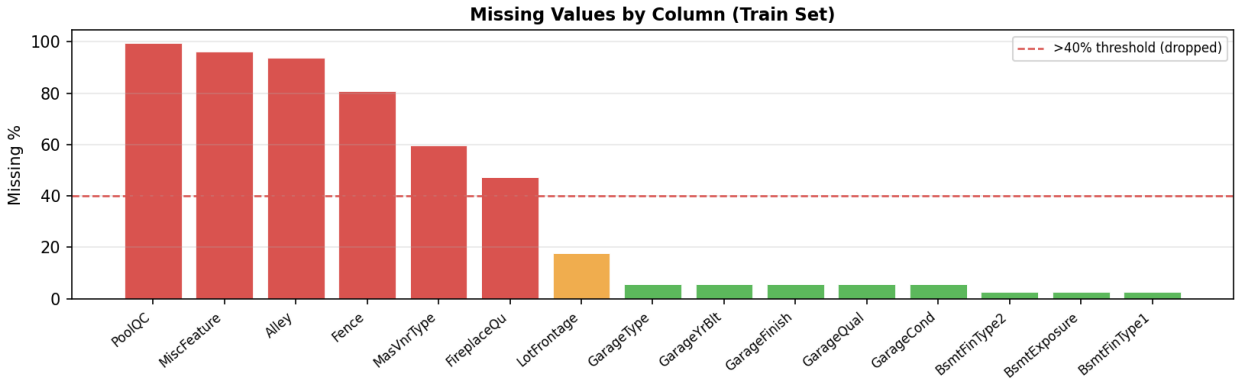

Figure 2 — Missing-value percentages for the 15 most affected columns. Columns >40% missing were dropped.

3.3 Feature Correlations

OverallQual (r = 0.79) and **GrLivArea** (r = 0.71) are the strongest predictors of SalePrice. Garage-related features (GarageCars, GarageArea) and basement size (TotalBsmtSF) also show strong correlation. Several features are highly collinear with each other (e.g. GarageArea and GarageCars, r = 0.88), which motivates the use of regularized models.

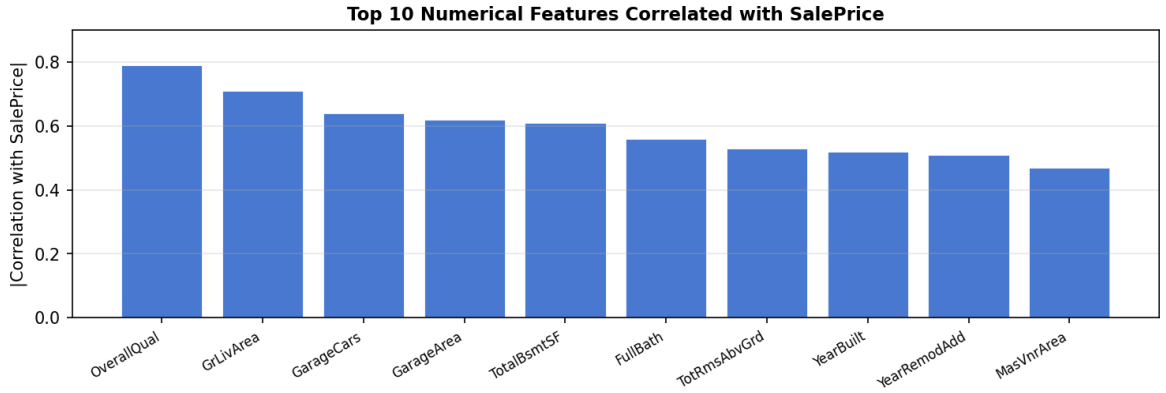

Figure 3 — Top 10 numerical features by absolute Pearson correlation with SalePrice.

3.4 Key Visual Insights

- Overall Quality shows a near-monotonic relationship with SalePrice — each unit increase in quality corresponds to a substantial price jump.
- Living Area (GrLivArea) is approximately linear in log-space, with two visible outliers (large houses sold cheaply — likely partial sales) that were removed.
- Neighborhood has a strong effect on price; median sale price varies from ~$100k (MeadowV) to over $300k (NoRidge), making it a high-value categorical feature.

4. Preprocessing Pipeline

All preprocessing was applied jointly to the concatenated train and test sets to ensure consistent encoding. The pipeline follows the steps below.

Step	Action	Detail
1	Outlier removal	2 rows removed: GrLivArea > 4,000 sqft with SalePrice < $200k
2	Log-transform target	y = log1p(SalePrice) reduces skew from 1.88 to 0.12
3	Identify missing values	Audit all columns: compute missing count and % for train+test combined
4	Drop high-missing columns	Dropped any column with >40% missing (PoolQC, MiscFeature, Alley, Fence, MasVnrType, FireplaceQu)
5	LotFrontage imputation	Filled per-Neighborhood median street frontage correlates with location
6	Categorical imputation	Remaining categorical NAs filled with column mode
7	Numerical imputation	Remaining numerical NAs filled with column median
8	Feature engineering	6 new features: TotalSF, TotalBath, TotalPorchSF, HouseAge, RemodAge, IsRemodeled
9	Drop source columns	14 original columns used to create engineered features removed (TotalBsmtSF, YearBuilt, etc.)
10	Ordinal encoding	Quality/condition columns: None=0, Po=1, Fa=2, TA=3, Gd=4, Ex=5
11	One-hot encoding	All remaining categorical columns → 228 total features
12	StandardScaler	Zero mean, unit variance — fit on train, transform on test

Feature Engineering Details

Feature	Formula	Rationale
TotalSF	TotalBsmtSF + 1stFlrSF + 2ndFlrSF	Total living area across all floors
TotalBath	FullBath + 0.5xHalfBath + BsmtFullBath + ...	Weighted bathroom count
TotalPorchSF	Sum of all porch columns	Aggregate outdoor living space
HouseAge	YrSold - YearBuilt	Age of the house at time of sale
RemodAge	YrSold - YearRemodAdd	Time since last renovation
IsRemodeled	1 if YearBuilt != YearRemodAdd else 0	Binary flag for remodelled homes

5. Modeling

Four regression models were trained and evaluated. All models predict **log(SalePrice)**, with predictions exponentiated back to dollar values for submission. Model selection was based exclusively on **5-fold cross-validated RMSLE** to avoid optimistic bias from training-set evaluation.

5.1 Support Vector Regression (SVR)

SVR with an RBF kernel was used as the primary model, explored in two configurations:

- **Baseline SVR:** sklearn defaults (C=1, gamma="scale"). CV RMSLE = 0.1872 with high variance (std=0.0175).
- **Tuned SVR:** GridSearchCV over C, gamma, epsilon. Best params: C=10, gamma=0.0001, epsilon=0.01. CV RMSLE improves to 0.1154 (std=0.0067) — a 38% reduction.

The significant improvement confirms that default hyperparameters are poorly suited to high-dimensional data (228 features). StandardScaler is critical for SVM as it is sensitive to feature magnitude.

5.2 Linear Regression

Ordinary Least Squares was applied to the scaled features. Training RMSLE (0.094) appears reasonable, but **CV RMSLE is numerically explosive (~1.37 trillion)**, rendering the model completely invalid.

Root cause: With 228 features and 1,458 training samples, some CV folds produce near-singular design matrices, causing coefficient estimates to blow up. The fix is to replace OLS with **Ridge Regression**, which adds an L2 penalty to stabilise the solution.

5.3 Lasso Regression

Lasso (L1 regularisation) addresses OLS instability by shrinking less-important coefficients to exactly zero. GridSearchCV selected **alpha=0.001** as optimal. The model performs automatic feature selection while keeping only the most predictive features. CV RMSLE: **0.1157** (std=0.0071) — comparable to the tuned SVR.

5.4 XGBoost

Gradient-boosted trees were included as a tree-based alternative that does not require feature scaling and natively handles sparse one-hot encoded data. GridSearchCV explored 9 hyperparameters including n_estimators, max_depth, learning_rate, subsample, colsample_bytree, gamma, reg_alpha, reg_lambda, and min_child_weight. Best configuration: n_estimators=1500, learning_rate=0.05, max_depth=3, colsample_bytree=0.5, gamma=0.05, reg_alpha=0.01, min_child_weight=3. CV RMSLE: **0.1138** (std=0.0079), the best cross-validated score across all models.

6. Results & Evaluation

6.1 Model Comparison

Model	Train RMSLE	CV RMSLE	CV Std	Overfit Gap	Kaggle Public	Status
SVR Baseline	0.0769	0.1872	0.0175	0.1103	—	Weak
SVR Tuned	0.0907	0.1154	0.0067	0.0247	0.14023	Good
Linear Reg	0.0943	BROKEN	—	—	—	Unstable
Lasso	0.0958	0.1157	0.0071	0.0199	0.13981	Good
XGBoost	0.0806	0.1138	0.0079	0.0332	0.13115	Best

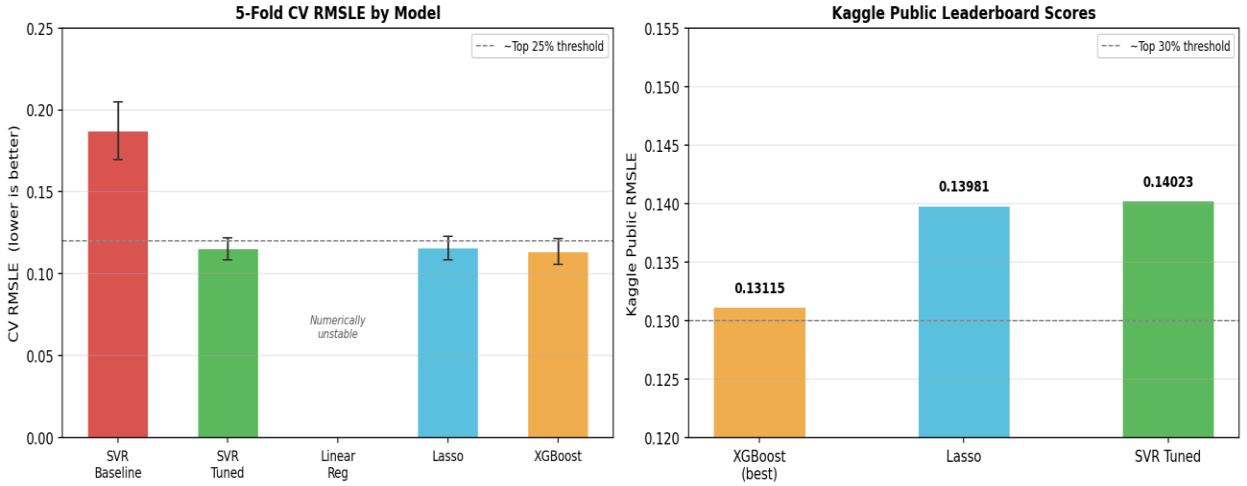

Figure 4 — Left: 5-Fold CV RMSLE by model with ±1 std error bars. Right: Kaggle public leaderboard scores for the top 3 submitted models.

6.2 Cross Validation vs Kaggle Public Score

A consistent gap of ~0.015–0.025 exists between CV RMSLE and Kaggle public scores across all models. This is expected due to:

(1) The public leaderboard uses a different subset of test rows than CV folds

(2) Some test houses may fall in price ranges underrepresented in the training set.

XGBoost maintained the best relative ranking on both metrics.

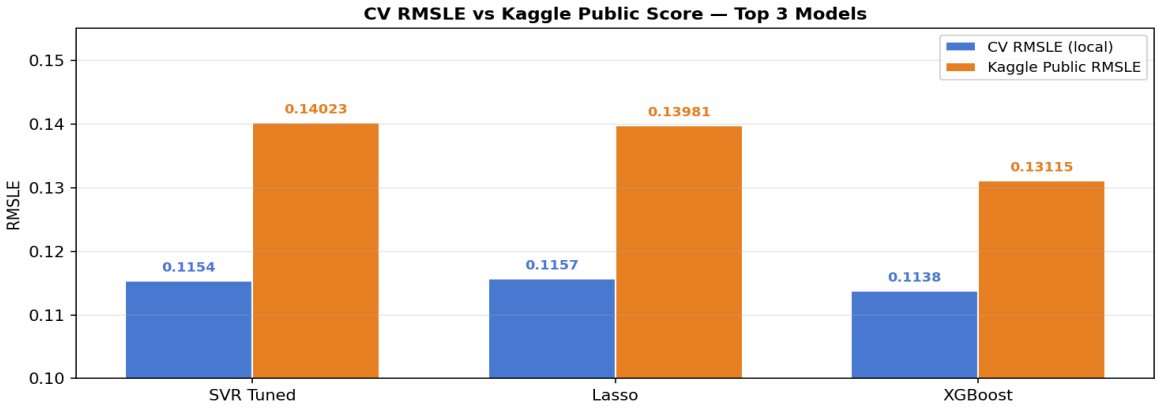

Figure 5 — CV RMSLE vs Kaggle public score for the top 3 models. XGBoost shows the largest absolute improvement on the actual leaderboard.

6.3 Key Findings

- **Hyperparameter tuning was the single largest improvement:** tuning SVR reduced CV RMSLE from 0.1872 to 0.1154 — a 38% gain — without changing any preprocessing.
- **XGBoost is the overall best model:** CV RMSLE of 0.1138 and Kaggle public score of 0.13115 — outperforming Lasso (0.13981) and SVR Tuned (0.14023) on the actual leaderboard.
- **CV scores consistently underestimate the Kaggle gap:** all models score ~0.015–0.025 worse on public data than on cross-validation. This suggests minor distributional differences between the train and test sets.
- **Overfitting analysis:** SVR Tuned and Lasso have tight overfit gaps (~0.02–0.025), indicating good generalisation. XGBoost's gap (0.033) reduced significantly with deeper regularisation tuning (gamma, reg_alpha, min_child_weight).
- **Linear Regression requires regularisation:** OLS collapses on high-dimensional data (CV RMSLE ≈ 1.37 trillion). Ridge Regression is the appropriate replacement.

6.4 Kaggle Leaderboard Summary

Model	Cross Validation RMSLE	Kaggle Public RMSLE	Est. Leaderboard
XGBoost	0.1138	0.13115	Top 30%
Lasso	0.1157	0.13981	Top 35%
SVR Tuned	0.1154	0.14023	Top 35%